



T.C.

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ

MÜHENDİSLİK FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ

Yapay Zeka Teknikleri Dersi Projesi

ABC Optimizasyon Algoritması

Emre KARATOP - 220610006

1. Giriş

1.1. Problem Tanımı

Perakende sektöründe karlılığı doğrudan etkileyen en kritik faktörlerden biri doğru fiyatlandırma stratejisidir. Bir ürünün fiyatı çok yüksek belirlendiğinde talep (satış adedi) düşmekte, fiyat çok düşük belirlendiğinde ise kar marjı azalmaktadır. Geleneksel yöntemlerde fiyatlandırma genellikle deneme-yanılma yoluyla veya yöneticilerin sezgilerine dayalı olarak yapılmaktadır. Ancak binlerce ürünün bulunduğu dinamik bir pazarda, her ürün için talep eğrisini manuel olarak analiz etmek ve "en tatlı noktayı" (sweet spot) bulmak insan kapasitesini aşan, karmaşık bir optimizasyon problemidir. Bu projede, fiyat ve satış adedi arasındaki doğrusal olmayan ilişki ele alınarak, matematiksel bir belirsizlik ortamında en doğru fiyatın nasıl bulunacağı problemi üzerinde durulmuştur.

1.2. Amaç

Bu çalışmanın temel amacı, sürü zekası tabanlı bir meta-sezgisel optimizasyon yöntemi olan Yapay Arı Kolonisi (Artificial Bee Colony - ABC) algoritmasını kullanarak, bir perakende ürününün toplam gelirini (Revenue) maksimize edecek optimum birim fiyatı tespit etmektir. Proje kapsamında:

1. Geçmiş satış verileri kullanılarak Fiyat-Talep ilişkisinin modellenmesi,
2. Türev gerektirmeyen bir arama algoritması ile en yüksek geliri sağlayan fiyatın simülasyon ortamında bulunması,
3. Mevcut ortalama fiyat ile algoritmanın önerdiği fiyatın karşılaştırılarak potansiyel gelir artışının raporlanması hedeflenmektedir.

2. Gelişme

2.1. Kullanılan Veri Seti : Online Retail

Projede, perakende satış verilerini içeren retail_price.csv veri seti kullanılmıştır. Veri seti, farklı kategorilerdeki ürünlerin tarihsel satış performanslarını içermektedir.

1. Analizin tutarlılığı açısından veri seti üzerinde şu ön işlemler yapılmıştır:
2. Veri setindeki en yüksek işlem hacmine sahip olan health7 (sağlık ürünleri kategorisi) ID'li ürün odak noktası olarak seçilmiştir.
3. Modelin girdisi olarak unit_price (birim fiyat), çıktısı olarak ise qty (satış adedi) öznitelikleri kullanılmıştır.
4. Verideki gürültüyü azaltmak ve optimizasyon algoritmasına bir zemin (amaç fonksiyonu) oluşturmak için, fiyat ve talep arasındaki ilişki 2. dereceden polinom regresyon yöntemiyle modellenmiştir ($Satış = a * Fiyat^2 + b * Fiyat + c$).

2.2. Kullanılan Algoritma : ABC

Problemin çözümü için Erciyes Üniversitesi'nden Derviş Karaboğa tarafından geliştirilen ve doğadaki arıların nektar arama davranışlarını taklit eden Yapay Arı Kolonisi (ABC) algoritması tercih edilmiştir. Gradient Descent gibi türev tabanlı yöntemlerin aksine, ABC algoritması yerel minimumlara takılmadan global (küresel) en iyi sonucu bulma yeteneğine sahiptir.

Algoritma üç temel fazda çalıştırılmıştır:

1. Görevli Arılar (Employed Bees): Rastgele fiyat seviyelerini (çözümleri) araştırır ve bu fiyatlardaki tahmini geliri hesaplar.
2. Gözcü Arılar (Onlooker Bees): Görevli arıların getirdiği "nektar" (gelir) bilgilerine göre, daha yüksek gelir potansiyeli olan fiyat aralıklarına yoğunlaşır (Exploitation).
3. Kaşif Arılar (Scout Bees): Belirli bir süre boyunca iyileştirilemeyen (limit aşımı) çözümleri terk eder ve arama uzayında rastgele yeni fiyatlar dener (Exploration).

3. Sonuç ve Değerlendirme

Yapılan simülasyon ve optimizasyon çalışması sonucunda, health7 ID'li ürün üzerinde elde edilen bulgular şöyledir:

- Mevcut Durum: Veri setindeki health7 ürününün geçmiş satış verilerine göre ortalama satış fiyatı 60.49 TL seviyesindedir.
- Optimizasyon Sonucu: Yapay Arı Kolonisi (ABC) algoritması, arama uzayındaki taramaları sonucunda ürünün birim fiyatının 97.48 TL seviyesine çıkarılmasını önermiştir.
- Değerlendirme: Algoritma, mevcut fiyatlandırmanın ürünün gerçek gelir potansiyelinin altında kaldığını tespit etmiştir. Fiyatın yaklaşık %61 oranında artırılması durumunda, satış adedinde yaşanacak olası düşüşe rağmen, birim başına artan kar marjı sayesinde toplam gelirin 187,764.15 TL seviyesine (tahmini maksimum) ulaşacağı öngörülmüştür.

Bu sonuç, söz konusu ürünün talebinin fiyata karşı nispeten esnek olmadığını (inelastic), yani müşterilerin bu ürün için daha yüksek bir bedel ödemeye istekli olduğunu göstermektedir. ABC algoritması, insan sezgisiyle fark edilmesi zor olan bu "eksik fiyatlandırma" durumunu başarılı bir şekilde ortaya çıkarmış ve işletme karlılığını artıracak optimum noktayı belirlemiştir.

Parametre	Mevcut Durum (Ortalama)	ABC Algoritması Sonucu (Optimize)	Değişim / Fark
Analiz Edilen Ürün	health7	health7	-
Birim Fiyat	60.49 TL	97.48 TL	+%61.15 (Artış)
Tahmini Toplam Gelir	(Mevcut Düzey)	187,764.15 TL	Maksimum Potansiyel
Strateji	Geleneksel Fiyatlandırma	Dinamik Optimizasyon	Kar Maksimizasyonu

4. Kaynak

- <https://www.kaggle.com/datasets/suddharshan/retail-price-optimization>

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. SINIF TANIMI: YAPAY ARI KOLONİSİ (ABC)
# -----
class ArtificialBeeColony:
    def __init__(self, objective_func, bounds, colony_size=30, n_iter=100,
limit=20):
        self.func = objective_func
        self.bounds = bounds # [min, max] arama aralığı
        self.colony_size = colony_size
        self.n_iter = n_iter
        self.limit = limit # Kaşif arı (scout) olma limiti

        # Koloniyi ikiye böl: Görevli (Employed) ve Gözcü (Onlooker)
        self.n_employed = colony_size // 2
        self.n_onlooker = colony_size // 2

        # Başlangıç: Rastgele yiyecek kaynakları (çözümler) üret
        self.foods = np.random.uniform(bounds[0], bounds[1], self.n_employed)
        self.trials = np.zeros(self.n_employed) # Her kaynağın kaç kez
denendiği

        # İlk en iyiyi bul
        self.best_solution = None
        self.best_fitness = float('inf')

    def optimize(self):
        for it in range(self.n_iter):
            # A. GÖREVLİ ARILAR FAZI (Employed Bees Phase)
            for i in range(self.n_employed):
                self.update_location(i)

            # B. GÖZCÜ ARILAR FAZI (Onlooker Bees Phase)
            # Fitness hesapla (Minimizasyon problemi olduğu için değeri ters
çeviriyoruz)
            # Amaç fonksiyonu negatif olduğu için (örn: -100), mutlak değeri
alıp olasılık hesaplıyoruz.
            fitness_vals = np.array([self.func(f) for f in self.foods])

            # Daha düşük değeri (daha negatif) daha iyi fitness demektir.
            # Rulet tekerleği için ters orantı kuralım:
            # (Basitlik için: En kötüden farklarını alıp normalize edelim)
            worst = fitness_vals.max()

```

```

        diffs = worst - fitness_vals + 1e-5 # +epsilon sıfıra bölmeyi
önler
        probs = diffs / np.sum(diffs)

        # Gözcüler olasılığa göre kaynak seçer
        for i in range(self.n_onlooker):
            selected_idx = np.random.choice(range(self.n_employed),
p=probs)
            self.update_location(selected_idx)

        # C. KAŞIF ARILAR FAZI (Scout Bees Phase)
        # Limiti aşan kaynakları terk et ve yenisini bul
        for i in range(self.n_employed):
            if self.trials[i] > self.limit:
                self.foods[i] = np.random.uniform(self.bounds[0],
self.bounds[1])
                self.trials[i] = 0

        # D. EN İYİYİ KAYDET
        current_vals = [self.func(f) for f in self.foods]
        min_val = min(current_vals)
        if min_val < self.best_fitness:
            self.best_fitness = min_val
            self.best_solution = self.foods[np.argmin(current_vals)]

    return self.best_solution, self.best_fitness

def update_location(self, i):
    # Rastgele bir komşu (k) seç, kendisi olmasın
    k = np.random.randint(0, self.n_employed)
    while k == i: k = np.random.randint(0, self.n_employed)

    # Yeni pozisyon formülü:  $v_i = x_i + \phi * (x_i - x_k)$ 
    phi = np.random.uniform(-1, 1)
    new_solution = self.foods[i] + phi * (self.foods[i] - self.foods[k])

    # Sınır kontrolü (Fiyat negatif veya aşırı yüksek olmasın)
    new_solution = np.clip(new_solution, self.bounds[0], self.bounds[1])

    # Greedy Selection: Yeni çözüm eskisinden iyiyse kabul et
    if self.func(new_solution) < self.func(self.foods[i]):
        self.foods[i] = new_solution
        self.trials[i] = 0
    else:
        self.trials[i] += 1

# -----
# 2. VERİ HAZIRLIĞI VE MODELLEME
# -----

```

```

# Veriyi oku
df = pd.read_csv('retail_price.csv')

# En popüler ürünü seç
top_product = df['product_id'].value_counts().idxmax()
product_df = df[df['product_id'] == top_product]

X_price = product_df['unit_price'].values
y_qty = product_df['qty'].values

# Talep Eğrisi (Polinom Regresyon 2. Derece)
# Qty = a*Price^2 + b*Price + c
coeffs = np.polyfit(X_price, y_qty, 2)
demand_model = np.poly1d(coeffs)

# Amaç Fonksiyonu (Objective Function)
# Geliri Maksimize etmek istiyoruz, Algoritmalar Minimize eder.
# Bu yüzden (-Revenue) döndürüyoruz.
def objective_function(price):
    pred_qty = demand_model(price)
    if pred_qty <= 0: return 1e10 # Satış yoksa çok yüksek ceza

    revenue = price * pred_qty
    return -revenue # Minimizasyon için negatif

# -----
# 3. ÇALIŞTIRMA
# -----
# Arama aralığı: Mevcut min fiyatın yarısı ile max fiyatın 1.5 katı arası
bounds = [X_price.min() * 0.5, X_price.max() * 1.5]

# ABC'yi Başlat
abc = ArtificialBeeColony(objective_function, bounds, colony_size=40,
n_iter=100)
best_price, best_neg_revenue = abc.optimize()

print(f"--- SONUÇLAR ({top_product}) ---")
print(f"Mevcut Ortalama Fiyat: {X_price.mean():.2f}")
print(f"ABC Önerilen Fiyat : {best_price:.2f}")
print(f"Beklenen Maks. Gelir : {-best_neg_revenue:.2f}")

```

```

--- SONUÇLAR (health7) ---
Mevcut Ortalama Fiyat: 60.49
ABC Önerilen Fiyat : 97.48
Beklenen Maks. Gelir : 187764.15

```